

Tutorial del Generador de Horarios a partir de un *Abstract Syntax Tree* en Common Lisp

Guía completa para la definición de dominios,
modelado matemático y ejecución de la metaheurística

Este documento describe el uso del generador de código Python desarrollado como parte de una tesis de grado. Cubre desde la instalación del entorno hasta la interpretación de los resultados, y puede emplearse para modelar cualquier tipo de problema de asignación de horarios.

Índice

1. Introducción	3
2. Requisitos previos	3
2.1. Software necesario	3
2.2. Instalación de SBCL	3
2.3. Instalación de Python	5
2.4. Obtención del código	5
3. Conceptos fundamentales	5
3.1. El Árbol de Sintaxis Abstracta (AST)	5
3.2. Tipos de cuantificadores de restricción	6
3.3. Operadores lógicos disponibles	6
4. Definición del dominio	7
4.1. Carga de la biblioteca	8
4.2. Definición de <i>mixins</i>	8
4.3. Definición de entidades	8
4.3.1. Entidades enumeradas	9
4.4. Definición de la entidad combinación	9
4.5. Definición de consultas	10
4.5.1. Consulta sin iteración	10
4.5.2. Consulta con iteración	10
4.6. Definición de restricciones	12
4.6.1. Restricción nppq (No Puede Pasar Que)	12
4.6.2. Restricción tqpq (Todo Que Pasa Que)	12
4.6.3. Restricción débil: minimizar	12
4.6.4. Restricciones blandas: tdpq y sbqp	13
4.7. Definición de datos concretos	13
5. Llamada al generador	14
5.1. Descripción de cada parámetro	15
5.2. Finalización del script	16
6. Ejecución	16
6.1. Paso 1: generar el archivo Python	16
6.2. Paso 2: ejecutar el algoritmo genético	17
7. Interpretación de la salida	17
7.1. Modelo de optimización	17
7.1.1. Lectura del modelo	18
7.2. Progreso del algoritmo genético	18
7.3. Horario propuesto y evaluación	18
8. Ejemplo completo: horario de defensas de tesis	19

9. Generalización a otros tipos de horarios	22
9.1. Lista de verificación antes de ejecutar	23
10.Referencia rápida de macros	24
11.Preguntas frecuentes	24
Conclusión	25

1. Introducción

El presente tutorial describe el uso del **Generador de Horarios**, una herramienta que permite definir un problema de asignación de horarios mediante un Árbol de Sintaxis Abstracta (AST) escrito en Common Lisp y genera automáticamente código Python con:

1. Las funciones de evaluación de cada restricción del problema.
2. El modelo matemático formal del problema de optimización.
3. Un algoritmo genético que construye y optimiza el horario.

La filosofía del sistema es **separar la descripción del problema de su resolución**: el usuario define qué entidades existen en su dominio, qué condiciones deben cumplirse y qué datos concretos hay; el generador produce el código ejecutable de forma automática.

El sistema ha sido diseñado para ser **genérico**: aunque el ejemplo que acompaña este tutorial corresponde a la asignación de defensas de tesis, la misma herramienta puede emplearse para horarios de clases, turnos de personal, asignación de salas, calendarios deportivos u cualquier otro problema de naturaleza similar.

2. Requisitos previos

2.1. Software necesario

Para utilizar el generador se necesitan los siguientes programas:

Steel Bank Common Lisp (SBCL) Intérprete de Common Lisp. Disponible en <https://www.sbcl.org/>. Versión mínima recomendada: 2.0.

Python 3.10 o superior Para ejecutar el código generado. Disponible en <https://www.python.org/>. No se requieren paquetes externos; la implementación utiliza únicamente la biblioteca estándar de Python.

2.2. Instalación de SBCL

La instalación varía según el sistema operativo.

Windows

1. Descargar el instalador desde <https://www.sbcl.org/platform-table.html> (seleccionar *Windows x64*).
2. Ejecutar el instalador y seguir los pasos predeterminados. SBCL se instala típicamente en `C:\Program Files\Steel Bank Common Lisp\`.

3. Para ejecutar `sbcl` desde PowerShell sin indicar la ruta completa, añadir el directorio de instalación al `PATH` del sistema (ejecutar una sola vez y reiniciar la terminal):

```
# PowerShell
[Environment]::SetEnvironmentVariable(
    "PATH",
    $env:PATH + ";C:\Program Files\Steel Bank Common Lisp",
    "User"
)
```

macOS

En macOS la forma más sencilla es utilizar *Homebrew*:

```
# Terminal (bash / zsh)
brew install sbcl
```

Alternativamente puede descargarse el binario precompilado desde <https://www.sbcl.org/platform-table.html> (seleccionar *macOS*). Tras la descarga, descomprimir y ejecutar el script de instalación incluido:

```
tar -xzf sbcl-<version>-x86-64-darwin-binary.tar.bz2
cd sbcl-<version>-x86-64-darwin
sudo sh install.sh
```

Linux

En distribuciones basadas en *Debian/Ubuntu*:

```
# bash
sudo apt-get update
sudo apt-get install sbcl
```

En distribuciones basadas en *Fedora/RHEL/Rocky Linux*:

```
sudo dnf install sbcl
```

En *Arch Linux*:

```
sudo pacman -S sbcl
```

En todos los casos, para verificar que la instalación fue exitosa:

```
sbcl --version
```

2.3. Instalación de Python

Python 3.10 o superior está disponible en <https://www.python.org/downloads/>. En macOS y Linux es frecuente que ya esté instalado; puede verificarse con:

```
python3 --version
```

En Windows, Python puede instalarse desde <https://www.python.org/> o desde la Microsoft Store. Durante la instalación se recomienda marcar la opción *Add Python to PATH* para poder invocarlo simplemente como `python` desde cualquier terminal.

2.4. Obtención del código

El código fuente se encuentra en el repositorio:

https://github.com/salmaf04/restrictions_language

Solo es necesaria la carpeta *generador-horarios/* del repositorio. Puede clonarse el repositorio completo y trabajar desde esa carpeta, o descargarse únicamente dicha carpeta. La estructura esperada del proyecto es la siguiente:

```
generador-horarios/  
+-- lib/  
|   +-- clases-macros.lisp      <- infraestructura del AST (no modificar)  
|   +-- generador.lisp         <- punto de entrada del generador  
|   +-- gen-utils.lisp  
|   +-- gen-ast-python.lisp  
|   +-- gen-ast-math.lisp  
|   +-- gen-serializar.lisp  
|   +-- gen-consultas.lisp  
|   +-- gen-restricciones.lisp  
|   +-- gen-visualizacion.lisp  
|   \-- gen-metaheuristica.lisp  
\-- mi-dominio/  
    \-- dominio.lisp           <- archivo que el usuario crea
```

Los archivos bajo *lib/* son la **biblioteca del generador** y no deben modificarse. El usuario únicamente necesita crear su propio archivo de dominio (véase la sección 4).

3. Conceptos fundamentales

3.1. El Árbol de Sintaxis Abstracta (AST)

El AST es la representación interna del problema. Cuando el usuario escribe definiciones en Common Lisp usando las macros del sistema, estas construyen automáticamente

objetos en memoria que forman un árbol. El generador recorre ese árbol y produce el código Python equivalente.

El usuario **nunca manipula el árbol directamente**: trabaja con las macros de alto nivel descritas en este documento.

3.2. Tipos de cuantificadores de restricción

El sistema reconoce cuatro tipos de cuantificadores para expresar restricciones, más dos operadores de optimización:

Cuadro 1: Cuantificadores del sistema

Macro	Nombre	Semántica
<code>def-nppq</code>	No Puede Pasar Que	Restricción dura. La condición nunca puede ser verdadera.
<code>def-tqpq</code>	Todo Que Pasa Que	Restricción dura. Toda entidad que satisface la premisa.
<code>def-tdpq</code>	Tampoco Debe Pasar Que	Restricción débil (blanda). Equivalente semántico a <code>def-nppq</code> .
<code>def-sbqp</code>	Sería Bueno Que Pasara	Restricción débil (blanda). Equivalente semántico a <code>def-tqpq</code> .
<code>def-minimizar</code>	Minimizar	Objetivo de optimización. Minimiza el máximo valor.
<code>def-maximizar</code>	Maximizar	Objetivo de optimización. Maximiza el mínimo valor.

Las restricciones **duras** (`nppq`, `tqpq`) reciben un peso alto (por defecto 1000) en la función objetivo. Las **blandas** (`tdpq`, `sbqp`, `minimizar`, `maximizar`) reciben un peso bajo (por defecto 1). La metaheurística minimiza la suma ponderada de violaciones.

3.3. Operadores lógicos disponibles

Para construir condiciones se dispone de los siguientes operadores:

Cuadro 2: Operadores lógicos y relacionales

Macro	Significado
<code>def-op-and e1 e2 ...</code>	Conjunción ($e_1 \wedge e_2 \wedge \dots$)
<code>def-op-or e1 e2 ...</code>	Disyunción ($e_1 \vee e_2 \vee \dots$)
<code>def-op-not e</code>	Negación ($\neg e$)
<code>def-op-igual a b</code>	Igualdad ($a = b$)
<code>def-op-distinto a b</code>	Desigualdad ($a \neq b$)
<code>def-op-mayor a b</code>	Mayor estricto ($a > b$)
<code>def-op-menor a b</code>	Menor estricto ($a < b$)
<code>def-op-mayor-igual a b</code>	Mayor o igual ($a \geq b$)
<code>def-op-menor-igual a b</code>	Menor o igual ($a \leq b$)

4. Definición del dominio

Todo el dominio se describe en un único archivo Lisp que el usuario crea. Este archivo debe:

1. Cargar la biblioteca del generador.
2. Definir los *mixins* (atributos compartidos).
3. Definir las entidades del dominio.
4. Definir la entidad combinación (la “fila” del horario).
5. Definir las consultas (auxiliares y con iteración).
6. Definir las restricciones.
7. Definir los datos concretos del problema.
8. Llamar a `generar-python`.

La estructura general del archivo es la siguiente:

Listing 1: Estructura del archivo de dominio

```
1 ;; 1. Cargar la biblioteca
2 (load (merge-pathnames "lib/clases-macros.lisp" (truename ".")))
3 (load (merge-pathnames "lib/generador.lisp" (truename ".")))
4
5 ;; 2. Mixins
6 (defclass* tiene-nombre () (nombre))
7
8 ;; 3. Entidades
9 (def-entidad recurso-a () (atributo1 atributo2))
10
11 ;; 4. Entidad combinación
12 (def-comb asignacion (identidad variable1 variable2))
13
14 ;; 5. Consultas
15 (def-consulta mi-consulta (arg1 arg2) ...)
16
17 ;; 6. Restricciones
18 (defvar restriccion-1 (def-nppq ...))
19
20 ;; 7. Datos concretos
21 (def-recurso-a inst-1 valor1 valor2)
22
23 ;; 8. Generar
24 (generar-python "salida/horario.py"
25   :entidad-horario 'asignacion
26   ...)
27
28 (sb-ext:exit)
```


Las subsecciones siguientes describen cada bloque en detalle.

4.1. Carga de la biblioteca

El archivo debe comenzar cargando los dos archivos de la biblioteca. La ruta es relativa al directorio desde el que se ejecutará SBCL, que debe ser la raíz del proyecto (*generador-horarios/*):

Listing 2: Carga de la biblioteca

```
1 (load (merge-pathnames "lib/clases-macros.lisp" (truename ".")))
2 (load (merge-pathnames "lib/generador.lisp"      (truename ".")))
```

4.2. Definición de *mixins*

Un *mixin* es un conjunto de atributos que pueden compartir varias entidades. Se define con la macro `defclass*`:

Listing 3: Sintaxis de `defclass*`

```
1 (defclass* nombre-mixin (padres...) (slot1 slot2 ...))
```

El ejemplo más común es un mixin con un atributo `nombre`:

Listing 4: Ejemplo de mixin

```
1 (defclass* tiene-nombre () (nombre))
```

Tras esta definición, cualquier entidad que herede de `tiene-nombre` dispondrá automáticamente del atributo `nombre` y de la función de acceso (`nombre entidad`).

4.3. Definición de entidades

Una entidad representa un tipo de objeto del dominio. Se define con `def-entidad`:

Listing 5: Sintaxis de `def-entidad`

```
1 (def-entidad nombre-entidad (padres-mixin...) (slot1 slot2 ...))
```

`nombre-entidad` Nombre del tipo de entidad (símbolo Lisp).

`(padres-mixin...)` Lista de *mixins* de los que hereda. Puede estar vacía: `()`.

`(slot1 slot2 ...)` Atributos propios de esta entidad. También puede estar vacía si la entidad solo tiene atributos heredados.

Ejemplo: definir un tipo `profesor` que tiene `nombre` (heredado del mixin) y `grado académico` (propio):

Listing 6: Definición de la entidad profesor

```
1 (def-entidad profesor (tiene-nombre) (grado))
```

Internamente, `def-entidad` genera:

- Una clase CLOS para las instancias concretas.
- Una clase CLOS para la referencia genérica (usada en restricciones).
- Una macro `def-profesor` para crear instancias.
- Funciones de acceso AST: `(nombre prof)`, `(grado prof)`.

4.3.1. Entidades enumeradas

Una **entidad enumerada** es aquella cuyas instancias son fijas y conocidas de antemano (como grados académicos, días de la semana o estados). Se define igual que cualquier entidad, pero generalmente con un único atributo identificador:

Listing 7: Entidad enumerada: grado académico

```
1 (def-entidad grado () (id))
2
3 (def-grado grado-lic "Lic")
4 (def-grado grado-msc "Msc")
5 (def-grado grado-dr "Dr")
```

4.4. Definición de la entidad combinación

La entidad combinación representa una **fila del horario**: qué identidad (p.ej., un tribunal, un grupo, un empleado) es asignada a qué recursos (p.ej., fecha, hora, aula). Se define con `def-comb`:

Listing 8: Sintaxis de `def-comb`

```
1 (def-comb nombre-combinacion (slot-identidad variable1
    variable2 ...))
```

slot-identidad El atributo que identifica de forma única *qué* se está asignando. Hay exactamente una instancia de la combinación por cada valor de este atributo.

variable1, variable2, ... Los atributos que el algoritmo genético decide: cuándo, dónde, en qué condiciones se realiza la asignación.

Ejemplo para un horario de defensas de tesis:

Listing 9: Entidad combinación para defensas de tesis

```
1 (def-comb asignacion (tribunal fecha momento local))
```

Aquí, **tribunal** es la identidad (hay una defensa por tribunal) y **fecha**, **momento**, **local** son las variables de decisión que el algoritmo optimizará.

4.5. Definición de consultas

Las consultas son expresiones con nombre y parámetros que pueden reutilizarse dentro de restricciones o de otras consultas. Se definen con **def-consulta** en dos variantes: con o sin iteración sobre las instancias del horario.

4.5.1. Consulta sin iteración

Una consulta sin iteración recibe argumentos explícitos y devuelve una expresión del AST construida con los operadores disponibles. Es útil para encapsular condiciones o sub-expresiones que se repiten en varias restricciones, aunque su uso es opcional: una restricción puede construir directamente su expresión sin necesidad de una consulta auxiliar.

Listing 10: Sintaxis de consulta sin iteración

```
1 (def-consulta nombre-consulta (arg1 arg2 ...)  
2   :comprueba  
3   <expresion-del-AST>)
```

Ejemplo: expresión que verifica si un profesor pertenece al tribunal de una asignación dada:

Listing 11: Consulta auxiliar: profesor en tribunal

```
1 (def-consulta profesor-en-tribunal (prof asig)  
2   :comprueba  
3   (def-op-or  
4     (def-op-igual (tutor      (tribunal asig)) prof)  
5     (def-op-igual (oponente   (tribunal asig)) prof)  
6     (def-op-igual (presidente (tribunal asig)) prof)  
7     (def-op-igual (vocal      (tribunal asig)) prof)  
8     (def-op-igual (secretario (tribunal asig)) prof)))
```

El acceso encadenado (**tutor (tribunal asig)**) navega los atributos del AST: primero obtiene el **tribunal** de la asignación **asig** y luego su atributo **tutor**. El generador produce el código de acceso equivalente en Python de forma automática.

Esta consulta puede usarse como subexpresión dentro de otras consultas o restricciones.

4.5.2. Consulta con iteración

Una consulta con iteración recorre uno o varios dominios de entidades, filtra con una condición y acumula un resultado numérico. Es el mecanismo principal para calcular

métricas sobre el horario completo: conteos, sumas, o cualquier agregado expresable con las operaciones disponibles.

Listing 12: Sintaxis de consulta con iteración

```

1 (def-consulta nombre-consulta (args-extra...))
2   :itera-sobre (var1 entidad1) (var2 entidad2) ...
3   :comprueba  <condicion-booleana>
4   :operacion   <tipo-de-acumulacion>
5   :devuelve    nombre-resultado)

```

:itera-sobre Define los bucles anidados. Cada par (**variable** **entidad**) recorre todas las instancias de ese tipo definidas en el dominio.

:comprueba Condición booleana que determina qué combinaciones de variables se acumulan en el resultado.

:operacion Tipo de acumulación:

- **suma** o **contar**: cuenta cuántas veces se cumple la condición.
- **contar-distintos-dias**: cuenta cuántos valores distintos del atributo **fecha** aparecen entre los elementos que cumplen la condición.

:devuelve Nombre descriptivo del resultado (informativo, no afecta al código generado).

Ejemplo: contar cuántas combinaciones de profesor y par de asignaciones presentan conflicto de horario:

Listing 13: Consulta con iteración: conflictos de horario

```

1 (def-consulta conflictos-de-horario ()
2   :itera-sobre (p profesor) (a1 asignacion) (a2 asignacion)
3   :comprueba
4     (def-op-and
5       (profesor-en-tribunal p a1)
6       (profesor-en-tribunal p a2)
7       (def-op-igual (fecha a1) (fecha a2))
8       (def-op-igual (momento a1) (momento a2))
9       (def-op-distinto (tribunal a1) (tribunal a2)))
10  :operacion suma
11  :devuelve n-conflictos)

```

Ejemplo: contar cuántos días distintos asiste un profesor concreto a lo largo del horario:

Listing 14: Consulta: días distintos de asistencia

```

1 (def-consulta dias-asistidos (prof)
2   :itera-sobre (asig asignacion)
3   :comprueba (profesor-en-tribunal prof asig)
4   :operacion contar-distintos-dias
5   :devuelve n-dias)

```

Las consultas con iteración son las que deben listarse en el parámetro `:consultas` de `generar-python`. Las consultas sin iteración se referencian automáticamente desde las que las utilizan y no necesitan declararse explícitamente en esa lista.

4.6. Definición de restricciones

Las restricciones expresan condiciones sobre el horario y se almacenan en variables globales con `defvar`. Cada restricción envuelve una expresión del AST, que puede ser una llamada a una consulta, una combinación de operadores lógicos o cualquier otra construcción válida. El generador traduce cada restricción a una función Python que devuelve un valor numérico: cuanto mayor sea ese valor, mayor es el incumplimiento.

4.6.1. Restricción nppq (No Puede Pasar Que)

Expresa que la expresión dada **no debe ocurrir o debe valer cero**. El generador produce una función cuyo resultado se multiplica por el peso de la restricción en la función objetivo; un valor de cero indica cumplimiento total.

Listing 15: Sintaxis de restricción nppq

```
1 (defvar nombre-restriccion
2   (def-nppq <expresion-del-AST>))
```

Ejemplo: prohibir cualquier conflicto de horario entre profesores:

Listing 16: Restricción nppq: sin conflictos de horario

```
1 (defvar restriccion-sin-conflictos-de-horario
2   (def-nppq (def-op-mayor conflictos-de-horario 0)))
```

4.6.2. Restricción tqpq (Todo Que Pasa Que)

Expresa que la expresión dada **debe cumplirse**. La función generada devuelve un valor positivo proporcional al grado de incumplimiento, y cero cuando la condición se satisface plenamente.

Listing 17: Sintaxis de restricción tqpq

```
1 (defvar nombre-restriccion
2   (def-tqpq <expresion-del-AST>))
```

4.6.3. Restricción débil: minimizar

Expresa un objetivo de optimización: minimizar el valor máximo de una consulta evaluada sobre cada instancia de un dominio de entidades. Es útil para equilibrar cargas o recursos entre los distintos elementos del problema.

Listing 18: Restricción débil: minimizar carga máxima

```

1 (defvar restriccion-minimizar-carga-maxima
2   (def-minimizar dias-asistidos profesor))

```

El primer argumento es la consulta a evaluar y el segundo es el tipo de entidad sobre el que se itera para calcular el máximo.

4.6.4. Restricciones blandas: tdpq y sbqp

def-tdpq (Tampoco Debe Pasar Que) es la versión blanda de **nppq**: expresa algo que preferiblemente no debe ocurrir, pero cuya violación tiene un coste menor en la función objetivo.

def-sbqp (Sería Bueno Que Pasara) es la versión blanda de **tqpq**: expresa algo que sería deseable que se cumpliera, sin que su incumplimiento sea crítico.

Ambas tienen la misma sintaxis que sus equivalentes duros y aceptan el mismo tipo de expresiones del AST:

Listing 19: Restricciones blandas

```

1 (defvar restriccion-blanda-1
2   (def-tdpq <expresion-que-se-prefiere-no-ocurra>))
3
4 (defvar restriccion-blanda-2
5   (def-sbqp <expresion-deseable>))

```

4.7. Definición de datos concretos

Una vez definidas las entidades, se crean las instancias concretas del problema usando las macros generadas automáticamente por **def-entidad**. La macro tiene la forma **def-<nombre-entidad>**:

Listing 20: Sintaxis general para instanciar entidades

```

1 (def-<nombre-entidad> nombre-instancia valor1 valor2 ...)

```

Los valores se asignan en orden: primero los atributos heredados de los *mixins* (en el orden en que se declararon los mixins), y luego los atributos propios.

Ejemplo completo de datos para el horario de tesis:

Listing 21: Datos concretos del dominio de tesis

```

1 ;; Grados (enumeración)
2 (def-grado grado-lic "Lic")
3 (def-grado grado-msc "Msc")
4 (def-grado grado-dr "Dr")
5

```

```

6 ;; Profesores: (nombre grado)
7 ;; Orden: slot heredado 'nombre' primero, luego slot propio
  'grado'
8 (def-profesor prof-piad "Alejandro Piad" grado-dr)
9 (def-profesor prof-suarez "Carlos Suarez" grado-msc)
10 (def-profesor prof-garcia "Maria Garcia" grado-dr)
11
12 ;; Estudiantes: (nombre)
13 (def-estudiante est-rodriiguez "Laura Rodriguez")
14 (def-estudiante est-fernandez "Carlos Fernandez")
15
16 ;; Tribunales: (estudiante tutor oponente presidente vocal
  secretario)
17 (def-tribunal trib-1
18   est-rodriiguez prof-piad prof-suarez prof-garcia prof-piad
    prof-suarez)
19
20 ;; Fechas disponibles: (dia mes)
21 (def-fecha fecha-1 1 6)
22 (def-fecha fecha-2 2 6)
23 (def-fecha fecha-3 3 6)
24
25 ;; Momentos disponibles: (hora)
26 (def-momento momento-manana 9)
27 (def-momento momento-mediiodia 11)
28 (def-momento momento-tarde 14)
29
30 ;; Locales disponibles: (nombre)
31 (def-local local-a "Sala A")
32 (def-local local-b "Sala B")

```

Importante: el orden de los argumentos en `def-<entidad>` debe coincidir exactamente con el orden en que se declararon los slots en la definición de la entidad (primero los heredados, luego los propios).

Todas las instancias se registran automáticamente en un repositorio interno. El generador las lee para construir las constantes del módulo Python y poblar los pools del algoritmo genético.

5. Llamada al generador

Una vez definido el dominio completo, se llama a la función `generar-python` para producir el archivo Python:

Listing 22: Llamada completa a `generar-python`

```

1 (generar-python "mi-dominio/horario.py"
2   ;; Entidad combinación (la fila del horario)

```

```

3  :entidad-horario      'asignacion
4
5  ;; Slot que identifica cada fila (hay exactamente una por
   valor)
6  :slot-identidad      'tribunal
7
8  ;; Slots que el algoritmo genético optimizará
9  :slots-variables      '(fecha momento local)
10
11 ;; Consultas con :itera-sobre (defvar, no defun)
12 :consultas            '(conflictos-de-horario dias-asistidos)
13
14 ;; Restricciones duras (nppq / tqpq)
15 :restricciones-duras
   '(restriccion-sin-conflictos-de-horario)
16
17 ;; Restricciones blandas (tdpq / sbqp / minimizar / maximizar)
18 :restricciones-blandas '(restriccion-minimizar-carga-maxima)
19
20 ;; Pesos del modelo matemático (opcionales, defaults: 1000 y
   1)
21 :peso-dura            1000
22 :peso-blanda          1)

```

5.1. Descripción de cada parámetro

:entidad-horario Símbolo de la entidad combinación definida con `def-comb`. El horario es una lista de instancias de este tipo.

:slot-identidad Nombre del atributo de la combinación que actúa como clave: hay exactamente una asignación por cada valor distinto de este slot. El algoritmo genético *no modifica* este slot.

:slots-variables Lista de atributos que el algoritmo genético puede cambiar. Cada uno se elige aleatoriamente de entre las instancias del tipo correspondiente que hayan sido definidas con `def-<entidad>`.

:consultas Lista de las variables globales que almacenan nodos `consulta-simple` con `:itera-sobre`. Estas son las que se convierten en funciones Python. Las consultas auxiliares (sin `:itera-sobre`, definidas como `defun`) se referencian automáticamente desde las que sí iteran.

:restricciones-duras Lista de variables globales con cuantificadores `nppq` o `tqpq`. Su violación tiene un coste alto en la función objetivo (`:peso-dura`).

:restricciones-blandas Lista de variables globales con cuantificadores `tdpq`, `sbqp`, `minimizar` o `maximizar`. Su violación tiene un coste bajo (`:peso-blanda`).

:peso-dura Peso numérico para las restricciones duras. Por defecto: 1000.

:peso-blanda Peso numérico para las restricciones blandas. Por defecto: 1.

5.2. Finalización del script

Al final del archivo de dominio, se añade `(sb-ext:exit)` para que SBCL termine limpiamente tras generar el archivo:

```
1 (sb-ext:exit)
```

6. Ejecución

6.1. Paso 1: generar el archivo Python

Situarse en la raíz del proyecto (*generador-horarios/*) y ejecutar SBCL con el archivo de dominio. Los comandos varían según el sistema operativo.

Windows (PowerShell)

Si SBCL fue añadido al PATH (véase sección 2):

```
cd "C:\ruta\al\generador-horarios"  
sbcl --script mi-dominio/dominio.lisp
```

Si no fue añadido al PATH, usar la ruta completa:

```
cd "C:\ruta\al\generador-horarios"  
& "C:\Program Files\Steel Bank Common Lisp\sycl.exe" --script mi  
-dominio/dominio.lisp
```

macOS y Linux (Terminal)

```
cd /ruta/al/generador-horarios  
sbcl --script mi-dominio/dominio.lisp
```

En todos los casos, un mensaje de confirmación indicará la ruta del archivo generado:

```
Python generado en: mi-dominio/horario.py
```

6.2. Paso 2: ejecutar el algoritmo genético

Windows (PowerShell)

```
python mi-dominio/horario.py
```

Si python no está en el PATH:

```
& "C:\Users\<usuario>\AppData\Local\Programs\Python\Python313\python.exe" 'mi-dominio/horario.py'
```

macOS y Linux (Terminal)

```
python3 mi-dominio/horario.py
```

7. Interpretación de la salida

7.1. Modelo de optimización

La primera sección impresa corresponde al modelo matemático abstracto:

```
=====
MODELO DE OPTIMIZACION
=====

INDICES Y CONJUNTOS:
T = conjunto de tribunals          (|T| = 3)
F = conjunto de fecha disponibles (|F| = 3)
M = conjunto de momento disponibles (|M| = 3)
L = conjunto de local disponibles  (|L| = 2)
P = conjunto de profesor           (|P| = 6)

VARIABLES DE DECISION:
Para cada tribunal t en T se asigna:
    fecha_t   in F   (fecha)
    momento_t in M   (momento)
    local_t   in L   (local)

FUNCION OBJETIVO:
minimizar z(h) donde:
    z(h) = 1000 * sum_i incumpl_i(h) [duras]
          +      1 * sum_j incumpl_j(h) [blandas]

RESTRICCIONES DURAS (nppq / tqpq):
R1 [SIMPLE / DURA (nppq) -- No Puede Pasar Que]:
    conflictos_de_horario(h) = 0
    donde:
```

```

conflictos_de_horario(h) = Sum_{p in P, a1 in A, a2 in A}
  [prof_en_tribunal(p,a1) /\ prof_en_tribunal(p,a2)
   /\ fecha(a1)=fecha(a2) /\ momento(a1)=momento(a2)
   /\ tribunal(a1) != tribunal(a2)]

RESTRICCIONES BLANDAS (minimizar):
R2 [Minimizar]:
  minimizar max_{x in P} dias_asistidos(h, x)

```

7.1.1. Lectura del modelo

Índices y conjuntos Muestra los tipos de entidades involucrados y cuántas instancias concretas hay de cada uno en el problema.

Variables de decisión Indica qué atributos se están decidiendo para cada valor del slot identidad. La notación v_t significa “el valor del atributo v asignado al tribunal t ”.

Función objetivo Fórmula de minimización con los pesos configurados.

Restricciones Para cada restricción se muestra su tipo, la expresión matemática que debe satisfacerse y, cuando aplica, la definición formal de las consultas que intervienen.

7.2. Progreso del algoritmo genético

Durante la ejecución se imprime el progreso cada 50 generaciones:

```

Generacion    0: costo inicial = 3000
Generacion   50: mejor costo    = 1000
Generacion  100: mejor costo    = 1
Generacion  150: mejor costo    = 1
...

```

Un costo de 0 indica que todas las restricciones duras se cumplen perfectamente y los objetivos de optimización han alcanzado su mínimo. Si el algoritmo encuentra una solución perfecta antes de agotar las generaciones, lo indica explícitamente.

7.3. Horario propuesto y evaluación

Al finalizar, se imprime el horario concreto elegido por la metaheurística y la evaluación de cada restricción:

```

=====
HORARIO PROPUESTO POR LA METAHEURISTICA
=====

Defensa 1:
TRIBUNAL : (estudiante=Laura Rodriguez, tutor=Alejandro Piad, ...)

```

```

FECHA      : (dia=2, mes=6)
MOMENTO    : 11
LOCAL      : Sala A

Defensa 2:
TRIBUNAL   : (estudiante=Carlos Fernandez, tutor=Maria Garcia, ...)
FECHA      : (dia=2, mes=6)
MOMENTO    : 14
LOCAL      : Sala B

=====
EVALUACION DE RESTRICCIONES
=====
Restricciones simples / duras (peso 1000):
  restriccion_sin_conflictos_de_horario: 0 incumplimientos -> costo:
    0
Restricciones debiles / blandas (peso 1):
  restriccion_minimizar_carga_maxima: 1 -> costo: 1
Costo total: 1
=====

```

Interpretación:

- *0 incumplimientos* en la restricción dura significa que ningún profesor debe asistir a dos defensas simultáneas.
- El valor 1 en la restricción blanda indica que algún profesor asiste al menos un día; dado el tamaño del problema esto es inevitable.
- El costo total de 1 (solo blandas) es la mejor solución posible para este conjunto de datos.

8. Ejemplo completo: horario de defensas de tesis

Esta sección presenta el archivo de dominio completo utilizado como caso de prueba del sistema.

Listing 23: Archivo completo: tesis/dominio.lisp

```

1 ;;
2 ;; =====
3 ;; DOMINIO: HORARIO DE DEFENSAS DE TESIS
4 ;; =====
5 ;; Ejecutar desde la carpeta generador-horarios/:
6 ;; sbcl --script tesis/dominio.lisp
7 ;; =====
8 ;; Cargar infraestructura
9 (load (merge-pathnames "lib/clases-macros.lisp" (truename ".")))

```

```

10 (load (merge-pathnames "lib/generador.lisp" (truename ".")))
11
12
13 ;; -- Mixins
14 -----
14 (defclass* tiene-nombre () (nombre))
15
16
17 ;; -- Entidades
18 -----
18 (def-entidad grado      () (id))
19 (def-entidad profesor   (tiene-nombre) (grado))
20 (def-entidad estudiante (tiene-nombre))
21 (def-entidad tribunal   ()
22   (estudiante tutor oponente presidente vocal secretario))
23 (def-entidad fecha      () (dia mes))
24 (def-entidad momento    () (hora))
25 (def-entidad local      (tiene-nombre))
26
27
28 ;; -- Entidad combinación
29 -----
29 (def-comb asignacion (tribunal fecha momento local))
30
31
32 ;; -- Consultas
33 -----
34
35 ;; Auxiliar: profesor pertenece al tribunal de la asignacion
35 (def-consulta profesor-en-tribunal (prof asig)
36   :comprueba
37   (def-op-or
38     (def-op-igual (tutor      (tribunal asig)) prof)
39     (def-op-igual (oponente   (tribunal asig)) prof)
40     (def-op-igual (presidente (tribunal asig)) prof)
41     (def-op-igual (vocal      (tribunal asig)) prof)
42     (def-op-igual (secretario (tribunal asig)) prof)))
43
44 ;; Cuenta pares de defensas con conflicto de horario
45 (def-consulta conflictos-de-horario ()
46   :itera-sobre (p profesor) (a1 asignacion) (a2 asignacion)
47   :comprueba
48   (def-op-and
49     (profesor-en-tribunal p a1)
50     (profesor-en-tribunal p a2)
51     (def-op-igual (fecha      a1) (fecha      a2))
52     (def-op-igual (momento    a1) (momento    a2))
53     (def-op-distinto (tribunal a1) (tribunal a2)))
54   :operacion suma

```

```

55 :devuelve n-conflictos)
56
57 ;; Cuenta dias distintos que asiste un profesor
58 (def-consulta dias-asistidos (prof)
59   :itera-sobre (asig asignacion)
60   :comprueba (profesor-en-tribunal prof asig)
61   :operacion contar-distintos-dias
62   :devuelve n-dias)
63
64
65 ;; -- Restricciones
66 -----
67 ;; DURA: ningun profesor en dos defensas al mismo tiempo
68 (defvar restriccion-sin-conflictos-de-horario
69   (def-nppq (def-op-mayor conflictos-de-horario 0)))
70
71 ;; BLANDA: minimizar la carga del profesor que mas dias asiste
72 (defvar restriccion-minimizar-carga-maxima
73   (def-minimizar dias-asistidos profesor))
74
75
76 ;; -- Datos concretos
77 -----
78 (def-grado grado-lic "Lic")
79 (def-grado grado-msc "Msc")
80 (def-grado grado-dr "Dr")
81
82 (def-profesor prof-piad "Alejandro Piad" grado-dr)
83 (def-profesor prof-suarez "Carlos Suarez" grado-msc)
84 (def-profesor prof-garcia "Maria Garcia" grado-dr)
85 (def-profesor prof-torres "Luis Torres" grado-msc)
86 (def-profesor prof-mendez "Ana Mendez" grado-dr)
87 (def-profesor prof-herrera "Pedro Herrera" grado-msc)
88
89 (def-estudiante est-rodriiguez "Laura Rodriguez")
90 (def-estudiante est-fernandez "Carlos Fernandez")
91 (def-estudiante est-lopez "Sofia Lopez")
92
93 (def-tribunal trib-1
94   est-rodriiguez prof-piad prof-suarez prof-garcia
95   prof-torres prof-mendez)
96 (def-tribunal trib-2
97   est-fernandez prof-garcia prof-herrera prof-piad
98   prof-suarez prof-torres)
99 (def-tribunal trib-3
100   est-lopez prof-mendez prof-torres prof-suarez
101   prof-herrera prof-garcia)

```

```

99
100 (def-fecha fecha-1 1 6)
101 (def-fecha fecha-2 2 6)
102 (def-fecha fecha-3 3 6)
103
104 (def-momento momento-manana 9)
105 (def-momento momento-mediiodia 11)
106 (def-momento momento-tarde 14)
107
108 (def-local local-a "Sala A")
109 (def-local local-b "Sala B")
110
111
112 ;; -- Generar
113 -----
113 (generar-python "tesis/horario.py"
114   :entidad-horario 'asignacion
115   :slot-identidad 'tribunal
116   :slots-variables '(fecha momento local)
117   :consultas '(conflictos-de-horario dias-asistidos)
118   :restricciones-duras
119     '(restriccion-sin-conflictos-de-horario)
120   :restricciones-blandas '(restriccion-minimizar-carga-maxima)
121   :peso-dura 1000
122   :peso-blanda 1)
123 (sb-ext:exit)

```

9. Generalización a otros tipos de horarios

El sistema no está ligado al dominio de defensas de tesis. Puede usarse para cualquier problema donde se deba asignar recursos a identidades minimizando conflictos. La siguiente tabla muestra algunos ejemplos:

Para adaptar el generador a un nuevo dominio, el proceso es siempre el mismo:

1. **Identificar la entidad combinación:** qué es lo que se está asignando (el elemento principal del horario) y qué atributos tiene.
2. **Separar identidad de variables:** determinar cuál atributo identifica unívocamente cada elemento (identidad) y cuáles son los que se asignan (variables).
3. **Definir las restricciones:** expresar en términos de consultas cuándo se produce un conflicto o una situación indeseable.
4. **Categorizar las restricciones:** decidir cuáles son duras (nunca pueden violarse) y cuáles son blandas (se prefiere no violarlas pero no es crítico).
5. **Proporcionar los datos:** definir las instancias concretas de cada tipo de entidad.

Cuadro 3: Ejemplos de aplicación del generador

Problema	Identidad	Variables	Restricciones típicas
Horario de clases	grupo de alumnos	día, hora, aula, profesor	Sin solapamiento de profesores; aula con capacidad suficiente
Turnos de personal	empleado	día, turno, puesto	Horas máximas semanales; descanso mínimo entre turnos
Calendarios deportivos	partido	jornada, hora, estadio	Equipos sin dos partidos en un mismo día; uso alternado del estadio
Asignación de aulas	evento	fecha, hora, sala	Sin colisión de salas; compatibilidad de capacidad

6. **Ejecutar el generador** y analizar los resultados.

9.1. Lista de verificación antes de ejecutar

Antes de llamar a `generar-python`, conviene verificar lo siguiente:

- ☐ La entidad combinación está definida con `def-comb` y contiene exactamente un slot de identidad más los slots variables.
- ☐ Cada slot variable tiene al menos dos instancias concretas definidas (de lo contrario el algoritmo no puede variar).
- ☐ Cada consulta pasada en `:consultas` fue definida con `:itera-sobre` (es un `defvar`, no un `defun`).
- ☐ Las restricciones pasadas en `:restricciones-duras` y `:restricciones-blandas` son variables globales (definidas con `defvar`).
- ☐ El directorio destino del archivo Python existe.
- ☐ SBCL se ejecuta desde la raíz del proyecto (`generador-horarios/`).

Cuadro 4: Macros principales del sistema

Macro / Función	Uso
<code>defclass* N () (s1 s2)</code>	Define mixin con atributos <code>s1</code> , <code>s2</code>
<code>def-entidad N (mix) (s)</code>	Define entidad heredando mixins, con slot <code>s</code>
<code>def-comb N (id v1 v2)</code>	Define entidad combinación
<code>def-N inst v1 v2</code>	Crea instancia de entidad <code>N</code>
<code>def-consulta N (a)</code>	Consulta auxiliar (retorna expresión AST)
<code>:comprueba e</code>	
<code>def-consulta N ()</code>	Consulta con iteración (devuelve contador)
<code>:itera-sobre ...</code>	
<code>def-nppq cond</code>	Restricción dura: condición nunca verdadera
<code>def-tqpq cond</code>	Restricción dura: condición siempre verdadera
<code>def-tdpq cond</code>	Restricción blanda (como <code>nppq</code>)
<code>def-sbqp cond</code>	Restricción blanda (como <code>tqpq</code>)
<code>def-minimizar consulta dominio</code>	Objetivo: minimizar máximo de consulta
<code>def-maximizar consulta dominio</code>	Objetivo: maximizar mínimo de consulta
<code>def-op-and e1 e2 ...</code>	Conjunción de expresiones
<code>def-op-or e1 e2 ...</code>	Disyunción de expresiones
<code>def-op-not e</code>	Negación
<code>def-op-igual a b</code>	Igualdad
<code>def-op-distinto a b</code>	Desigualdad
<code>def-op-mayor a b</code>	$a > b$
<code>def-op-menor a b</code>	$a < b$
<code>(slot entidad)</code>	Acceso a un atributo de una entidad en el AST
<code>generar-python archivo &key ...</code>	Genera el archivo Python

10. Referencia rápida de macros

11. Preguntas frecuentes

¿Puedo tener más de una entidad combinación? No directamente. El algoritmo genético maneja una única lista de asignaciones. Si el problema requiere múltiples tipos de asignaciones, considere modelarlo con una sola entidad combinación que tenga más atributos variables.

¿Qué ocurre si no se encuentran soluciones perfectas? El algoritmo genético devuelve la mejor solución encontrada, aunque su costo sea mayor que cero. Esto puede

indicar que el problema es sobredeterminado (más restricciones duras de las que los datos permiten satisfacer simultáneamente), que el número de generaciones es insuficiente, o que los parámetros del algoritmo necesitan ajuste. Los parámetros `POP_SIZE`, `GENERACIONES`, `PROB_CRUCE`, `PROB_MUTACION` y `TORNEO_K` pueden modificarse directamente en el archivo Python generado.

¿Puedo añadir restricciones manualmente al Python generado? Sí. El archivo Python generado es código ordinario que puede editarse. Sin embargo, se recomienda añadir las restricciones al dominio Lisp y regenerar, ya que de lo contrario los cambios manuales se perderán en la próxima generación.

¿Cómo ajusto los pesos de las restricciones? A través de los parámetros `:peso-dura` y `:peso-blanda` de `generar-python`. Si se necesita que ciertas restricciones blandas tengan mayor prioridad que otras, una alternativa es usar distintos valores de `:peso-blanda` en llamadas separadas (creando dos grupos de restricciones con pesos distintos) y sumar los costos.

¿El orden de los slots en `def-entidad` importa? Sí, y es crítico. Los argumentos de la macro `def-<entidad>` deben proporcionarse en el mismo orden en que se declararon: primero los atributos heredados de los mixins (en el orden en que se listaron los mixins), y luego los atributos propios.

¿Puedo usar el generador sin SBCL? El generador está escrito en Common Lisp estándar (ANSI CL). En principio puede ejecutarse con cualquier implementación de Common Lisp, aunque las pruebas se han realizado exclusivamente con SBCL.

Conclusión

El generador de horarios proporciona un flujo de trabajo claro y reproducible para modelar problemas de asignación:

1. **Definir el dominio** en Lisp: mixins, entidades, combinación, consultas, restricciones y datos.
2. **Generar el Python** con `sbcl -script dominio.lisp`.
3. **Ejecutar la metaheurística** con `python horario.py`.
4. **Analizar el modelo matemático** y el horario propuesto.

La separación entre la descripción del problema (Lisp) y su resolución (Python generado) permite iterar rápidamente sobre el modelo: modificar restricciones, añadir datos o cambiar pesos sin necesidad de reescribir el código de optimización.